

Module 3 — Class 4 Assignment: Feature Selection

Type-it-yourself exercise

UN AI/ML Mentorship

Module 3 — Class 4 Assignment: Feature Selection

Topic: Feature selection — mutual information, RFE, model comparison

Goal: Open a **fresh Google Colab notebook** and type the code yourself by following the steps below. Do NOT copy-paste from the working notebook. Typing it yourself is how you learn the syntax.

How to submit

1. Open Google Colab → File → New Notebook.
 2. Type the code from each step below. Add a short # comment on EVERY cell so we know you understand it.
 3. Save the notebook as `Module<X>_Class<Y>_<YourName>_Assignment.ipynb`.
 4. Push it to your team repo at `module-<X>/class_<Y>/submissions/<YourName>/.`
-

Step 0 — Load and prepare

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
url = 'https://raw.githubusercontent.com/IBM/telco-customer-churn-on-icp4d/master/data/Telco-Customer-Churn.csv'
df = pd.read_csv(url)
df['TotalCharges'] = pd.to_numeric(df['TotalCharges'], errors='coerce').fillna(0)
df['Churn_bin'] = df['Churn'].map({'No': 0, 'Yes': 1})
```

Step 1 — Build a feature set with numeric + one-hot encoded categorical

What to do: Mix tenure, MonthlyCharges, TotalCharges + dummies for Contract.

Type this in a new cell:

```
num = df[['tenure', 'MonthlyCharges', 'TotalCharges', 'SeniorCitizen']]
cat = pd.get_dummies(df[['Contract', 'PaymentMethod']], drop_first=True)
X = pd.concat([num, cat], axis=1)
y = df['Churn_bin']
print('X shape:', X.shape)
print('Features:', list(X.columns))
```

Expected output:

X shape: (7043, ~9-10 features)

Step 2 — Mutual information scores for all features

What to do: Higher MI = more informative for predicting churn.

Type this in a new cell:

```
from sklearn.feature_selection import mutual_info_classif
mi = mutual_info_classif(X, y, random_state=42)
mi_scores = pd.Series(mi, index=X.columns).sort_values(ascending=False)
mi_scores.round(4)
```

Expected output:

tenure is usually highest, followed by Contract_Two year / MonthlyCharges.

Step 3 — Bar chart of MI scores

What to do: Visualise which features matter most.

Type this in a new cell:

```
plt.figure(figsize=(8, 5))
mi_scores.plot(kind='barh', color='steelblue')
plt.title('Mutual information with Churn')
plt.xlabel('MI score')
plt.tight_layout()
plt.show()
```

Expected output:

A horizontal bar chart with features ranked by MI.

Step 4 — RFE — pick the top 3 features

What to do: Use LogisticRegression as the base estimator.

Type this in a new cell:

```
from sklearn.feature_selection import RFE
from sklearn.linear_model import LogisticRegression
from sklearn.preprocessing import StandardScaler
X_scaled = StandardScaler().fit_transform(X)
rfe = RFE(LogisticRegression(max_iter=1000), n_features_to_select=3)
rfe.fit(X_scaled, y)
result = pd.DataFrame({'feature': X.columns, 'kept': rfe.support_, 'rank': rfe.ranking_})
result.sort_values('rank')
```

Expected output:

Table showing which 3 features were kept (rank=1).

Step 5 — Compare accuracy: all features vs top 3

What to do: Train a LogisticRegression on each and compare test accuracy.

Type this in a new cell:

```
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
X_tr, X_te, y_tr, y_te = train_test_split(X_scaled, y, test_size=0.2, random_state=42)
m_all = LogisticRegression(max_iter=1000).fit(X_tr, y_tr)
print('All features:', round(accuracy_score(y_te, m_all.predict(X_te)), 4))
idx = np.where(rfe.support_)[0]
m_top = LogisticRegression(max_iter=1000).fit(X_tr[:, idx], y_tr)
print('Top 3:', round(accuracy_score(y_te, m_top.predict(X_te[:, idx])), 4))
```

Expected output:

Both accuracies around 0.78-0.80; top-3 is usually very close to all-features.

Checklist before you submit

- ☐ All cells run without error
- ☐ Every code cell has a # comment in your own words
- ☐ Outputs are visible (you saved AFTER running)
- ☐ File name is correct: Module<X>_Class<Y>_<YourName>_Assignment.ipynb

Deadline: before next class.